

PLANO DE DESENVOLVIMENTO — FitCoach Pro

Versão: 1.0

Data: 17/06/2026

Referências: PRD v1.0 · SPEC v1.0

Abordagem: Web-first → Mobile quando web estiver robusta

PRINCÍPIOS DESTE PLANO

1. **Web primeiro, mobile depois** — O dashboard web e toda a infraestrutura de backend são construídos e validados antes de uma linha de código mobile ser escrita.
2. **Etapa concluída = gate aprovado** — Cada etapa tem um checklist de conclusão. A próxima etapa só começa quando o gate anterior está 100% verde.
3. **Banco de dados é fundação** — Nenhuma tela é construída sem as tabelas, RLS e seeds da etapa correspondente estarem prontos e testados.
4. **Deploy contínuo desde o dia 1** — Cada etapa termina com código em produção (Vercel + Supabase). Nunca acumular deploy.
5. **SPEC guia o aceite** — Cada tarefa de desenvolvimento referencia a SPEC correspondente. "Pronto" significa

critérios de aceite verificados, não "funciona na minha máquina".

VISÃO GERAL DAS FASES

FASE 0 – Infraestrutura e Setup	(Semana 1)
FASE 1 – Autenticação Web	(Semana 1-2)
FASE 2 – Gestão de Alunos	(Semana 2-3)
FASE 3 – Banco de Exercícios	(Semana 3)
FASE 4 – Criação e Gestão de Treinos	(Semana 4-5)
FASE 5 – Avaliações Físicas	(Semana 5-6)
FASE 6 – Financeiro (Faturas)	(Semana 6-7)
FASE 7 – Arquivos	(Semana 7)
FASE 8 – Dashboard e Relatórios	(Semana 8)
FASE 9 – Notificações e Automações Web	(Semana 8-9)
FASE 10 – White-label e Configurações	(Semana 9)
FASE 11 – Testes, QA e Hardening Web	(Semana 10)

GATE: WEB APROVADA

FASE 12 – Setup Mobile	(Semana
------------------------	---------

11)		
FASE 13 – Autenticação Mobile		(Semana 11)
FASE 14 – Home e Frequência		(Semana 12)
FASE 15 – Treinos (Visualização)		(Semana 12-13)
FASE 16 – Execução do Treino		(Semana 13-14)
FASE 17 – Avaliações e Progresso Mobile		(Semana 14-15)
FASE 18 – Faturas e Arquivos Mobile		(Semana 15)
FASE 19 – Notificações Push		(Semana 15-16)
FASE 20 – White-label Mobile		(Semana 16)
FASE 21 – Testes, QA e Deploy Mobile		(Semana 17)

Duração estimada total: 17 semanas

Duração Web: 10 semanas

Duração Mobile: 7 semanas

FASE 0 – INFRAESTRUTURA E SETUP

Objetivo: Ter o ambiente completo funcionando antes de qualquer feature.

Duração: 2–3 dias

SPECs relacionadas: Nenhuma (pré-requisito técnico)

ETAPA 0.1 — Repositórios GitHub

Tarefas

- ☐ Criar organização ou conta no GitHub para o projeto
- ☐ Criar repositório `fitcoach-web` (Next.js) — privado
- ☐ Criar repositório `fitcoach-mobile` (Expo) — privado
- ☐ Criar repositório `fitcoach-supabase` (migrations + edge functions) — privado
- ☐ Configurar `.gitignore` adequado em cada repositório
- ☐ Definir estratégia de branches: `main` (produção), `develop` (staging), `feature/*`
- ☐ Proteger branch `main`: exigir PR + aprovação antes de merge
- ☐ Criar arquivo `README.md` em cada repositório com instruções de setup local

Gate 0.1

- ☐ 3 repositórios criados e acessíveis
 - ☐ Branch protection configurada
 - ☐ README com instruções claras
-

ETAPA 0.2 — Projeto Supabase

Tarefas

- [] Criar projeto no Supabase (escolher região: `sa-east-1` — São Paulo)
- [] Anotar: `Project URL`, `anon key`, `service_role key`
- [] Instalar Supabase CLI localmente: `npm install -g supabase`
- [] Inicializar projeto: `supabase init` dentro de `fitcoach-supabase`
- [] Vincular projeto local ao remoto: `supabase link --project-ref SEU_ID`
- [] Criar pasta `supabase/migrations/` com arquivo `.gitkeep`
- [] Habilitar extensão `uuid-oss` no Supabase Dashboard
- [] Configurar e-mail de autenticação (SMTP ou Supabase built-in)
- [] Definir URL do site nas configurações de Auth: `https://fitcoachpro.vercel.app`

Gate 0.2

- [] `supabase status` retorna projeto vinculado
- [] Painel Supabase acessível
- [] Extensão `uuid-oss` ativa

ETAPA 0.3 — Projeto Next.js

Tarefas

- [] Criar projeto: `npx create-next-app@latest fitcoach-web --typescript --tailwind --eslint --app --src-dir`

- [] Instalar dependências base:

```
@supabase/supabase-js
@supabase/ssr
@radix-ui/react-* (componentes acessíveis)
lucide-react (ícones)
clsx
tailwind-merge
react-hook-form
zod
```

- [] Configurar `tailwind.config.ts` com paleta de cores do design system (PRD § 11.1)
- [] Criar estrutura de pastas conforme SPEC: `src/app/`, `src/components/`, `src/lib/`, `src/hooks/`
- [] Criar arquivo `.env.local` com variáveis do Supabase
- [] Criar `src/lib/supabase/client.ts` e `src/lib/supabase/server.ts`
- [] Criar `src/middleware.ts` (esqueleto — proteção de rotas)
- [] Testar `npm run dev` — servidor local funcionando

Gate 0.3

- [] `npm run dev` sem erros
- [] `npm run build` sem erros

- ☐ `npm run lint` sem warnings
-

ETAPA 0.4 – Deploy Inicial no Vercel

Tarefas

- ☐ Criar conta/projeto no Vercel
- ☐ Conectar repositório `fitcoach-web` ao Vercel
- ☐ Configurar variáveis de ambiente no Vercel (Supabase URL e keys)
- ☐ Configurar domínio de preview automático para branch `develop`
- ☐ Fazer primeiro deploy da branch `main` (página em branco/placeholder)
- ☐ Confirmar que build na Vercel passa sem erros

Gate 0.4

- ☐ URL de produção acessível (ex: `fitcoach-web.vercel.app`)
 - ☐ Deploy automático funcionando ao fazer push para `main`
 - ☐ Preview deploy funcionando ao abrir PR
-

ETAPA 0.5 – Design System Base

Tarefas

- [] Instalar e configurar `shadcn/ui`: `npx shadcn-ui@latest init`
- [] Adicionar componentes base: `button`, `input`, `label`, `card`, `badge`, `dialog`, `toast`, `dropdown-menu`, `select`, `tabs`, `table`
- [] Criar `src/components/ui/` com os componentes customizados
- [] Criar página de playground em `/dev/design` (só em development) listando todos os componentes
- [] Definir e documentar tokens: cores, tipografia, espaçamento no `tailwind.config.ts`
- [] Criar componente `<Logo />` com fallback para texto
- [] Criar componente `<PageHeader />` para cabeçalhos de página do dashboard
- [] Criar componente `<EmptyState />` para estados vazios

Gate 0.5

- [] Página `/dev/design` exibe todos os componentes sem erros
- [] Design system visualmente consistente com a paleta definida no PRD

FASE 1 — AUTENTICAÇÃO WEB

Objetivo: Personal consegue criar conta, confirmar e-mail e fazer login no dashboard.

Duração: 3–4 dias

SPECs: AUTH-01, AUTH-02, AUTH-06

ETAPA 1.1 – Banco: Tabela `personals` e Trigger

Tarefas

- ☐ Criar migration
`20260617000001_create_personals.sql`:
 - Tabela `personals` com todos os campos (PRD § 5)
 - Função `set_updated_at()` reutilizável
 - Trigger `personals_updated_at`
 - Trigger `on_personal_signup` → cria registro em `personals` ao signup com `role='personal'`
- ☐ Rodar migration: `supabase db push`
- ☐ Habilitar RLS na tabela `personals`
- ☐ Criar políticas RLS: `personal` vê e edita apenas seus próprios dados
- ☐ Testar no Supabase Studio: inserir `personal` manualmente e verificar RLS

Gate 1.1

- ☐ Migration aplicada sem erros
- ☐ RLS ativa — query sem auth retorna 0 registros
- ☐ Trigger cria registro em `personals` ao criar usuário com role `personal`

ETAPA 1.2 – Tela de Cadastro do Personal

Tarefas

- [] Criar rota `/register` com layout público
- [] Criar componente `<RegisterForm />` com campos: nome, e-mail, CREF, senha, confirmar senha
- [] Validação com `zod`: e-mail válido, senha 8+ chars com letra maiúscula e número
- [] Integrar `supabase.auth.signUp()` com metadata `{ role: 'personal', name }`
- [] Estado de loading no botão durante submit
- [] Após sucesso: redirecionar para `/register/confirm` com instrução de verificar e-mail
- [] Tratar erro de e-mail duplicado: exibir mensagem sem revelar existência da conta (SPEC-AUTH-01 AC-03)
- [] Criar página `/register/confirm` com instrução e botão "Reenviar e-mail"

Gate 1.2

- [] SPEC-AUTH-01 AC-01 a AC-07 verificados manualmente
 - [] Cadastro cria registro em `auth.users` + `personals`
 - [] E-mail de confirmação chega na caixa do testador
-

ETAPA 1.3 – Tela de Login do Personal

Tarefas

- [] Criar rota `/login` com layout público
- [] Criar componente `<LoginForm />` com campos: e-mail, senha (com toggle de visibilidade)
- [] Integrar `supabase.auth.signInWithPassword()`
- [] Após login bem-sucedido: redirecionar para `/dashboard`
- [] Mensagem de erro genérica para credenciais inválidas (SPEC-AUTH-02 AC-02)
- [] Implementar contador de tentativas falhas com bloqueio de 15 minutos após 5 tentativas (SPEC-AUTH-02 AC-03)
- [] Link "Esqueci minha senha" → `/forgot-password`
- [] Criar rota `/forgot-password` com campo de e-mail e `supabase.auth.resetPasswordForEmail()`
- [] Criar rota `/reset-password` para redefinição via link do e-mail

Gate 1.3

- [] SPEC-AUTH-02 AC-01 a AC-06 verificados
 - [] Login redireciona corretamente para `/dashboard`
 - [] Credenciais erradas exibem mensagem genérica (não revela qual campo)
 - [] Fluxo de recuperação de senha funciona de ponta a ponta
-

ETAPA 1.4 – Middleware e Proteção de Rotas

Tarefas

- [] Completar `src/middleware.ts`: verificar sessão em todas as rotas `/dashboard/*`
- [] Redirecionar para `/login` se não autenticado
- [] Redirecionar para `/dashboard` se usuário autenticado tentar acessar `/login` ou `/register`
- [] Verificar que usuário autenticado tem registro em `personals` (evitar acesso de alunos pelo dashboard)
- [] Criar layout `src/app/(auth)/layout.tsx` para páginas públicas (login, register)
- [] Criar layout `src/app/(dashboard)/layout.tsx` com sidebar e header

Gate 1.4

- [] Acesso a `/dashboard` sem sessão redireciona para `/login`
 - [] Após login, `/login` redireciona automaticamente para `/dashboard`
 - [] Sessão persiste após fechar e reabrir o navegador
-

ETAPA 1.5 – Layout do Dashboard (Shell)

Tarefas

- [] Criar componente `<Sidebar />` com links: Dashboard, Alunos, Exercícios, Treinos, Faturas, Arquivos, Configurações
- [] Criar componente `<TopBar />` com: saudação, ícone de notificações (placeholder), avatar do personal
- [] Sidebar responsiva: colapsável em telas menores
- [] Criar página `/dashboard` com conteúdo placeholder ("Em construção")
- [] Implementar logout: `supabase.auth.signOut()` + redirect para `/login` (SPEC-AUTH-06)
- [] Avatar do personal: foto se cadastrada, ou iniciais como fallback

Gate 1.5

- [] Dashboard abre com sidebar e topbar sem erros
- [] Logout funciona e limpa sessão
- [] Layout responsivo funciona em 1280px e 768px

FASE 2 – GESTÃO DE ALUNOS

Objetivo: Personal consegue cadastrar, editar, convidar, listar e gerenciar alunos.

Duração: 5–6 dias

SPECs: ALU-01 a ALU-05, AUTH-03

ETAPA 2.1 — Banco: Tabela `students`

Tarefas

- ☐ Criar migration
`20260617000002_create_students.sql`:
 - Tabela `students` com todos os campos do PRD
 - Índices: `personal_id`, `user_id`, `unique (email, personal_id)`
 - Trigger `students_updated_at`
- ☐ Criar políticas RLS para `students`:
 - Personal gerencia seus próprios alunos
 - Aluno vê apenas seus próprios dados (preparar para futuro)
- ☐ Criar seed com 3 alunos fictícios vinculados a um personal de teste
- ☐ Rodar e verificar no Supabase Studio

Gate 2.1

- ☐ Migration aplicada sem erros
 - ☐ RLS: personal A não vê alunos do personal B (teste manual no Studio)
 - ☐ Seed funciona e dados aparecem corretamente
-

ETAPA 2.2 — Listar Alunos

Tarefas

- ☐ Criar rota `/dashboard/students` com Server Component
- ☐ Criar `StudentService.list(personalId)` em `src/lib/services/student.service.ts`
- ☐ Criar componente `<StudentCard />` com: foto/avatar, nome, objetivo, último treino (placeholder), status do plano
- ☐ Implementar busca por nome (filtro client-side na lista)
- ☐ Implementar filtro por status: Todos / Ativos / Inativos
- ☐ Estado vazio: ilustração + botão "Cadastrar primeiro aluno"
- ☐ Badge visual para plano vencido (vermelho) e aluno inativo (cinza)

Gate 2.2

- ☐ SPEC-ALU-01 AC-01 a AC-07 verificados
 - ☐ Busca por nome funciona em tempo real
 - ☐ RLS confirma: personal só vê seus alunos
-

ETAPA 2.3 – Cadastrar Aluno

Tarefas

- ☐ Criar rota `/dashboard/students/new` com formulário

- ☐ Campos: nome*, e-mail*, telefone, data nascimento, objetivo (select), observações médicas, valor plano, dia vencimento, dias de treino (checkboxes S T Q Q S S D)
- ☐ Validação com `zod` : campos obrigatórios, formato e-mail, unicidade de e-mail dentro do personal
- ☐ Server Action `createStudentAction()` chamando `StudentService.create()`
- ☐ Verificar limite de alunos do plano Free (máx 5) antes de salvar (SPEC-ALU-02 AC-07)
- ☐ Modal de upgrade ao atingir limite (SPEC-PROF-04 AC-02)
- ☐ Após salvar: redirect para `/dashboard/students/[id]` com toast de sucesso

Gate 2.3

- ☐ SPEC-ALU-02 AC-01 a AC-07 verificados
- ☐ Cadastro cria aluno no banco vinculado ao personal correto
- ☐ Limite de 5 alunos no plano Free funciona

ETAPA 2.4 – Editar Aluno

Tarefas

- ☐ Criar rota `/dashboard/students/[id]/edit`
- ☐ Carregar dados atuais do aluno no formulário (Server Component)

- [] Mesmo formulário do cadastro, com campo e-mail bloqueado após convite enviado (SPEC-ALU-03 AC-01)
- [] Server Action `updateStudentAction()`
- [] Toast de confirmação "Alterações salvas" após sucesso

Gate 2.4

- [] SPEC-ALU-03 AC-01 a AC-04 verificados
 - [] E-mail não editável após convite enviado
 - [] Alterações refletidas na lista imediatamente
-

ETAPA 2.5 — Perfil do Aluno (Página Detalhada)

Tarefas

- [] Criar rota `/dashboard/students/[id]` com abas: Dados, Treinos, Avaliações, Faturas
- [] Aba Dados: resumo do aluno + botões de ação (Editar, WhatsApp, Inativar)
- [] Aba Treinos: lista de planos (placeholder "Nenhum treino" por enquanto)
- [] Aba Avaliações: placeholder
- [] Aba Faturas: placeholder
- [] Botão WhatsApp: abre `https://wa.me/[numero]` em nova aba

- [] Botão Inativar: dialog de confirmação antes de inativar (SPEC-ALU-04 AC-01)
- [] Botão Reativar disponível em alunos inativos

Gate 2.5

- [] SPEC-ALU-05 AC-01 a AC-06 verificados
- [] Inativar exige confirmação
- [] Aluno inativado aparece com badge "Inativo"

ETAPA 2.6 – Convite de Acesso (Edge Function)

Tarefas

- [] Criar Edge Function `supabase/functions/invite-student/index.ts`
- [] Função cria `auth.user` com senha temporária e metadata `{ role: 'student' }`
- [] Atualiza `students.user_id` e `invite_sent_at`
- [] Enviar e-mail com credenciais (usar Supabase built-in email ou Resend)
- [] Criar botão "Enviar convite" no perfil do aluno (sem convite enviado)
- [] Botão "Reenviar convite" se `invite_sent_at` preenchido mas `first_login_at` nulo
- [] Exibir status: "Convite pendente", "Acessou em DD/MM" (SPEC-AUTH-03 AC-04)

- ☐ Deploy da Edge Function: `supabase functions deploy invite-student`

Gate 2.6

- ☐ SPEC-AUTH-03 AC-01 a AC-06 verificados
 - ☐ Convite chega no e-mail com credenciais corretas
 - ☐ Status do convite atualiza no dashboard após envio
-

FASE 3 — BANCO DE EXERCÍCIOS

Objetivo: Personal consegue criar, editar, organizar e buscar exercícios com vídeos.

Duração: 3–4 dias

SPECs: EXE-01 a EXE-04

ETAPA 3.1 — Banco: Tabela `exercises` e Storage

Tarefas

- ☐ Criar migration
`20260617000003_create_exercises.sql`:
 - Tipo enum `muscle_group`
 - Tabela `exercises` com todos os campos
 - Índices: `personal_id`, `muscle_group`
 - Trigger `exercises_updated_at`

- [] RLS: personal gerencia seus exercícios; aluno vê exercícios do seu personal
- [] Criar Storage Bucket `exercise-videos` (público, até 100MB, apenas vídeos)
- [] Criar Storage Bucket `exercise-thumbnails` (público, até 5MB, apenas imagens)
- [] Políticas de Storage: personal faz upload apenas em pasta `{personal_id}/`

Gate 3.1

- [] Migration aplicada; enum `muscle_group` criado
- [] RLS funciona: personal A não vê exercícios do personal B
- [] Upload de arquivo de vídeo no bucket funciona via Supabase Studio

ETAPA 3.2 — Listar e Buscar Exercícios

Tarefas

- [] Criar rota `/dashboard/exercises`
- [] Criar `ExerciseService.list()` com filtro por `personal_id`
- [] Grid de exercícios com: thumbnail, nome, grupo muscular
- [] Busca por nome (tempo real)
- [] Filtro por grupo muscular (chips/badges selecionáveis)

- ☐ Toggle para ver exercícios arquivados (SPEC-EXE-03 AC-03)
- ☐ Estado vazio com botão "Criar primeiro exercício"

Gate 3.2

- ☐ SPEC-EXE-04 AC-01 a AC-04 verificados
- ☐ Busca e filtro funcionam corretamente

ETAPA 3.3 – Criar e Editar Exercício

Tarefas

- ☐ Criar rota `/dashboard/exercises/new`
- ☐ Campos: nome*, grupo muscular*, instruções, vídeo, thumbnail
- ☐ Vídeo: tabs para escolher entre "Upload", "YouTube URL" ou "Instagram URL"
- ☐ Upload de vídeo: barra de progresso, limite 100MB (SPEC-EXE-01 AC-04)
- ☐ YouTube URL: extrair thumbnail automaticamente com `https://img.youtube.com/vi/{VIDEO_ID}/hqdefault.jpg`
- ☐ Preview da thumbnail antes de salvar
- ☐ Criar rota `/dashboard/exercises/[id]/edit` com mesmo formulário
- ☐ Alerta ao salvar edição se exercício está em treinos publicados (SPEC-EXE-02 AC-03)

- ☐ Botão "Arquivar" no formulário de edição com confirmação

Gate 3.3

- ☐ SPEC-EXE-01 AC-01 a AC-07 verificados
 - ☐ SPEC-EXE-02 AC-01 a AC-03 verificados
 - ☐ Upload de vídeo funciona com barra de progresso
 - ☐ Thumbnail do YouTube é extraída automaticamente
-

FASE 4 – CRIAÇÃO E GESTÃO DE TREINOS

Objetivo: Personal consegue criar treinos completos, adicionar exercícios com configurações, publicar e gerenciar o plano do aluno.

Duração: 6–7 dias

SPECs: TRE-01 a TRE-08 (parte web), EXET-06 (evolução – visualização web)

ETAPA 4.1 – Banco: Tabelas de Treino

Tarefas

- ☐ Criar migration
20260617000004_create_workout_plans.sql :
 - Tabela workout_plans

- Tabela `workout_exercises`
- Índices necessários
- Triggers `updated_at`
- ☐ Criar migration
`20260617000005_create_workout_sessions.sql`:
 - Tabela `workout_sessions`
 - Tabela `session_exercise_logs`
 - Coluna gerada `duration_seconds`
 - Índices
- ☐ RLS para todas as 4 tabelas:
 - Personal gerencia planos dos seus alunos
 - Aluno vê apenas planos publicados dos seus treinos
 - Aluno gerencia suas próprias sessões
- ☐ Criar VIEW `workout_plan_stats` (contagem de sessões + última execução)
- ☐ Criar VIEW `student_weekly_frequency`
- ☐ Seed: criar 2 planos fictícios com exercícios para aluno de teste

Gate 4.1

- ☐ Todas as migrations aplicadas sem erros
 - ☐ VIEWS retornam dados corretos
 - ☐ RLS: aluno não vê treinos em rascunho
`(is_published = false)`
-

ETAPA 4.2 – Lista de Treinos do Aluno (Visão do Personal)

Tarefas

- [] Na aba "Treinos" do perfil do aluno, listar planos com:
 - Nome, descrição, status (Rascunho / Publicado / Arquivado)
 - Total de sessões e data da última execução (da VIEW `workout_plan_stats`)
- [] Botões: Editar treino, Publicar/Despublicar, Arquivar, Duplicar
- [] Botão "Novo treino" no topo
- [] Ordenação drag-and-drop para definir `display_order`
- [] Seção separada para Treinos Extras

Gate 4.2

- [] Lista exibe corretamente planos com contagem de sessões
- [] Status visual diferencia Rascunho / Publicado / Arquivado

ETAPA 4.3 – Criar e Editar Treino

Tarefas

- ☐ Criar rota
/dashboard/students/[id]/workouts/new
- ☐ Criar rota
/dashboard/students/[id]/workouts/[planId]
(editor)
- ☐ Campos do treino: nome*, descrição, tipo (Regular / Extra)
- ☐ Seção de exercícios: lista vazia com botão "Adicionar exercício"
- ☐ Modal/drawer para adicionar exercício:
 - Busca no banco de exercícios do personal
 - Ao selecionar: campos de séries, reps, carga, intervalo (segundos), instruções específicas
- ☐ Cada exercício na lista exibe: thumbnail, nome, configurações, botão remover
- ☐ Drag-and-drop para reordenar exercícios da lista (SPEC-TRE-03)
- ☐ Auto-save a cada 30 segundos (SPEC-AVA-01 AC-08 – padrão a seguir)
- ☐ Salvar rascunho vs Publicar (SPEC-TRE-04 AC-01)
- ☐ Botão "Publicar" desabilitado se sem exercícios (SPEC-TRE-04 AC-01)

Gate 4.3

- ☐ SPEC-TRE-01 AC-01 a AC-06 verificados
- ☐ SPEC-TRE-02 AC-01 a AC-06 verificados
- ☐ SPEC-TRE-03 AC-01 a AC-03 verificados

- [] Exercício adicionado aparece com thumbnail e configurações corretas
-

ETAPA 4.4 – Publicar, Arquivar e Duplicar Treino

Tarefas

- [] Implementar `WorkoutService.publish(planId)` – muda `is_published = true`
- [] Implementar `WorkoutService.unpublish(planId)` – muda `is_published = false`
- [] Implementar `WorkoutService.archive(planId)` com verificação de sessão recente
- [] Implementar `WorkoutService.duplicate(planId, targetStudentId)` – copia plano e exercícios
- [] Dialog de confirmação para despublicar e arquivar
- [] Dialog de seleção de aluno destino ao duplicar
- [] Toast de sucesso/erro para cada ação

Gate 4.4

- [] SPEC-TRE-04 AC-01 a AC-05 verificados
 - [] SPEC-TRE-05 AC-01 a AC-04 verificados
 - [] SPEC-TRE-06 AC-01 a AC-04 verificados
 - [] Duplicata cria plano em rascunho com todos os exercícios copiados
-

ETAPA 4.5 – Visualização de Evolução (Web)

Tarefas

- ☐ Criar rota
`/dashboard/students/[id]/workouts/[planId]/evolution`
- ☐ Selecionar exercício (dropdown)
- ☐ Gráfico de linha: eixo X = data das sessões, eixo Y = carga utilizada
- ☐ Instalar biblioteca de gráficos: `recharts` ou `chart.js`
- ☐ Comparativo: "Última sessão: X kg (+Y kg vs anterior)"
- ☐ Lista de sessões com data, duração e nota de feedback

Gate 4.5

- ☐ SPEC-EXET-06 AC-01 a AC-06 verificados (visão web)
 - ☐ Gráfico renderiza corretamente com dados de seed
-

FASE 5 – AVALIAÇÕES FÍSICAS

Objetivo: Personal consegue registrar avaliações completas com medidas e fotos; aluno verá histórico no mobile (implementação posterior).

Duração: 4–5 dias

SPECs: AVA-01 a AVA-05

ETAPA 5.1 – Banco: Tabelas de Avaliação

Tarefas

- ☐ Criar migration
20260617000006_create_assessments.sql :
 - Tabela `physical_assessments` com todos os campos de medidas e dobras
 - Coluna gerada `bmi`
 - Tabela `student_progress`
 - Índices e triggers
- ☐ RLS: personal gerencia avaliações; aluno vê apenas as suas
- ☐ Criar Storage Bucket `assessment-photos` (privado — signed URLs)
- ☐ Políticas de Storage para fotos de avaliação
- ☐ Seed: 2 avaliações para aluno de teste

Gate 5.1

- ☐ Migration aplicada; IMC calculado automaticamente
 - ☐ Bucket privado criado — URL direta retorna 403
 - ☐ Signed URL funciona para acesso temporário
-

ETAPA 5.2 – Formulário de Avaliação

Tarefas

- ☐ Criar rota
`/dashboard/students/[id]/assessments/new`
- ☐ Formulário dividido em seções:
 - **Data e Composição Corporal:** data, peso, altura → IMC calculado ao vivo, % gordura, massa magra, massa gorda
 - **Medidas Corporais (cm):** todos os campos em grid organizado
 - **Dobras Cutâneas (mm):** protocolo Pollock 7 dobras
 - **Fotos:** upload de até 3 fotos (frente, lateral, costas) com preview
 - **Laudos:** textarea para observações
- ☐ Upload de fotos com preview antes de salvar
- ☐ Auto-save a cada 30 segundos (SPEC-AVA-01 AC-08)
- ☐ Validação de tamanho de foto: máx 10MB (SPEC-AVA-01 AC-06)
- ☐ Após salvar: redirect para histórico de avaliações do aluno

Gate 5.2

- ☐ SPEC-AVA-01 AC-01 a AC-08 verificados
- ☐ IMC calculado ao vivo ao digitar peso e altura
- ☐ 3 fotos fazem upload para Storage com URLs assinadas

ETAPA 5.3 – Histórico e Detalhes de Avaliação

Tarefas

- ☐ Criar rota `/dashboard/students/[id]/assessments`
- ☐ Lista de avaliações em ordem cronológica decrescente
- ☐ Card de cada avaliação: data, peso, % gordura, IMC
- ☐ Comparativo automático entre última e penúltima (SPEC-AVA-02 AC-04)
- ☐ Criar rota `/dashboard/students/[id]/assessments/[assessmentId]`
- ☐ Detalhe completo com todas as medidas, fotos (via signed URL) e laudo
- ☐ Botão Editar (SPEC-AVA-04) e Excluir (SPEC-AVA-05) com confirmação

Gate 5.3

- ☐ SPEC-AVA-02 AC-01 a AC-04 verificados
 - ☐ Fotos carregam via signed URL (link temporário, não URL pública)
 - ☐ Exclusão remove foto do Storage e registro do banco
-
-

FASE 6 — FINANCEIRO

(FATURAS)

Objetivo: Personal consegue criar, gerenciar e acompanhar faturas dos alunos.

Duração: 4–5 dias

SPECs: FAT-01 a FAT-07 (exceto FAT-07 que é mobile)

ETAPA 6.1 — Banco: Tabela `invoices`

Tarefas

- ☐ Criar migration
`20260617000007_create_invoices.sql`:
 - Tipo enum `invoice_status`
 - Tabela `invoices` com todos os campos
 - Índices: `student_id`, `personal_id`, `status`, `due_date`
 - Trigger `invoices_updated_at`
- ☐ RLS: personal gerencia faturas; aluno vê apenas as suas
- ☐ Seed: faturas em diferentes status para aluno de teste

Gate 6.1

- ☐ Migration aplicada com enum correto
- ☐ RLS funciona para personal e aluno

ETAPA 6.2 – Criar e Listar Faturas

Tarefas

- ☐ Criar rota `/dashboard/invoices` – visão geral de todas as faturas
- ☐ Totalizadores no topo: Recebido no mês, A receber, Vencido (SPEC-FAT-06 AC-02)
- ☐ Tabela de faturas com colunas: aluno, valor, vencimento, referência, status, ações
- ☐ Filtros: por aluno, por status, por período
- ☐ Ordenação: por vencimento (padrão), por aluno, por valor
- ☐ Criar rota `/dashboard/invoices/new` com formulário
- ☐ Campos: aluno (select), *valor* (pré-preenchido com valor padrão do aluno), vencimento*, método de pagamento, link, chave PIX, referência, observações
- ☐ Aba de faturas dentro do perfil do aluno também

Gate 6.2

- ☐ SPEC-FAT-01 AC-01 a AC-05 verificados
- ☐ SPEC-FAT-06 AC-01 a AC-04 verificados
- ☐ Totalizadores calculam corretamente

ETAPA 6.3 – Ações sobre Faturas

Tarefas

- ☐ Botão "Marcar como paga" → dialog com campo data real do pagamento (SPEC-FAT-02)
- ☐ Botão "Cancelar fatura" → dialog de confirmação (SPEC-FAT-05)
- ☐ Botão "Lembrar via WhatsApp" → abre WhatsApp com mensagem pré-formatada (SPEC-FAT-04)
- ☐ Registrar no banco que lembrete foi enviado (`reminder_sent_at`)
- ☐ Badge visual: Pendente (amarelo), Pago (verde), Vencido (vermelho), Cancelado (cinza)

Gate 6.3

- ☐ SPEC-FAT-02 AC-01 a AC-05 verificados
 - ☐ SPEC-FAT-04 AC-01 a AC-03 verificados
 - ☐ SPEC-FAT-05 AC-01 a AC-04 verificados
-

ETAPA 6.4 – Automação: Vencimento de Faturas

Tarefas

- ☐ Criar Edge Function `supabase/functions/overdue-invoices/index.ts`
- ☐ Lógica: busca faturas `pending` com `due_date < hoje` → atualiza para `overdue`

- [] Cria notificação in-app para o aluno (registra na tabela `notifications` – criar tabela primeiro)
- [] Configurar cron job no Supabase (`pg_cron`) para rodar diariamente às 01h00
- [] Testar manualmente: `supabase functions invoke overdue-invoices`

Gate 6.4

- [] SPEC-FAT-03 AC-01 a AC-04 verificados
 - [] Edge Function executa sem erro
 - [] Fatura com vencimento ontem muda para `vencido` após execução
-

FASE 7 – ARQUIVOS

Objetivo: Personal consegue enviar arquivos para alunos específicos ou para todos.

Duração: 2–3 dias

SPECs: ARQ-01 a ARQ-04 (parte web)

ETAPA 7.1 – Banco: Tabela `files` e Storage

Tarefas

- [] Criar migration `20260617000008_create_files.sql`

- ☐ RLS: personal gerencia seus arquivos; aluno vê arquivos do seu personal
- ☐ Criar Storage Bucket `student-files` (privado, máximo 50MB, PDF e imagens)
- ☐ Políticas de Storage

Gate 7.1

- ☐ Migration aplicada
 - ☐ Bucket privado criado
-

ETAPA 7.2 – Upload e Gestão de Arquivos

Tarefas

- ☐ Criar rota `/dashboard/files`
- ☐ Botão "Enviar arquivo" → modal com campos: nome, categoria (select), destinatário (todos ou aluno específico), upload
- ☐ Barra de progresso durante upload (SPEC-ARQ-01 AC-05)
- ☐ Lista de arquivos: nome, categoria, destinatário, data, tamanho, botões (excluir, reenviar notificação)
- ☐ Filtro por aluno e categoria
- ☐ Exclusão com confirmação e remoção do Storage (SPEC-ARQ-04)

Gate 7.2

- [] SPEC-ARQ-01 AC-01 a AC-05 verificados
 - [] SPEC-ARQ-02 AC-01 a AC-04 verificados
 - [] SPEC-ARQ-04 AC-01 a AC-03 verificados
-
-

FASE 8 — DASHBOARD HOME E RELATÓRIOS

Objetivo: Home do dashboard com visão geral real do personal e relatório de frequência.

Duração: 3–4 dias

SPECs: ALU-06, ALU-07, FRQ-02, FRQ-03

ETAPA 8.1 — Home do Dashboard (Dados Reais)

Tarefas

- [] Substituir placeholders por dados reais:
 - Total de alunos ativos
 - Treinos executados hoje e na semana (da tabela `workout_sessions`)
 - Receita mensal (soma de faturas pagas no mês corrente)
- [] Seção "Atividade Recente": últimas 10 sessões de todos os alunos com nome, treino e horário

- [] Seção "Planos vencendo": alunos com `plan_expires_at` nos próximos 7 dias
- [] Seção "Aniversariantes": alunos com `birth_date` no mês corrente
- [] Implementar saudação com horário: "Bom dia/Boa tarde/Boa noite, [Nome]!"
- [] Loading skeletons enquanto dados carregam

Gate 8.1

- [] Todos os cards exibem dados reais do banco
- [] Atividade recente atualiza ao executar treino no seed

ETAPA 8.2 — Relatório de Frequência

Tarefas

- [] Criar rota `/dashboard/students/[id]/frequency`
- [] Calendário de presença das últimas 4 semanas
- [] Totais: treinos no mês, taxa de frequência vs dias configurados
- [] Alerta visual se sem treinar há 5+ dias
- [] Filtro por período: 1 semana, 1 mês, 3 meses
- [] Criar página `/dashboard/reports/frequency` :
 - Lista todos os alunos com frequência semanal/mensal
 - Destaque para alunos inativos (5+ dias)
 - Ranking dos mais frequentes (SPEC-ALU-07)

Gate 8.2

- ☐ SPEC-ALU-06 AC-01 a AC-04 verificados
 - ☐ SPEC-FRQ-02 AC-01 a AC-04 verificados
 - ☐ SPEC-ALU-07 AC-01 a AC-04 verificados
-

FASE 9 – NOTIFICAÇÕES E AUTOMAÇÕES WEB

Objetivo: Sistema de notificações in-app funcionando e automações de alerta configuradas.

Duração: 3–4 dias

SPECs: NOT-01 a NOT-06, FRQ-03

ETAPA 9.1 – Banco: Tabelas de Notificações

Tarefas

- ☐ Criar migration
20260617000009_create_notifications.sql:
 - Tipo enum notification_recipient
 - Tabela notifications
 - Tabela push_tokens (preparar para mobile)
 - Índices e RLS

Gate 9.1

- ☐ Migration aplicada
 - ☐ RLS: usuário vê apenas suas notificações
-

ETAPA 9.2 – Central de Notificações In-app (Web)

Tarefas

- ☐ Ícone de sino no TopBar com badge de contagem não lidas
- ☐ Dropdown com lista de notificações
- ☐ Marcar como lida ao clicar (SPEC-NOT-02 AC-04)
- ☐ "Marcar todas como lidas" (SPEC-NOT-02 AC-05)
- ☐ Notificações são inseridas no banco pelas Edge Functions
- ☐ Usar Supabase Realtime para atualizar badge sem reload

Gate 9.2

- ☐ SPEC-NOT-02 AC-01 a AC-06 verificados
 - ☐ Badge atualiza em tempo real ao receber nova notificação
-

ETAPA 9.3 – Automação: Alerta de Inatividade

Tarefas

- ☐ Criar Edge Function
`supabase/functions/inactivity-alert/index.ts`
- ☐ Usar VIEW `inactive_students_alert` para encontrar alunos inativos há 5+ dias
- ☐ Criar notificação in-app para o personal (SPEC-FRQ-03)
- ☐ Evitar duplicatas: não renotificar se notificação gerada nos últimos 5 dias (SPEC-FRQ-03 AC-04)
- ☐ Configurar cron: diariamente às 08h00
- ☐ Criar VIEW `inactive_students_alert` no banco

Gate 9.3

- ☐ SPEC-FRQ-03 AC-01 a AC-05 verificados
 - ☐ Edge Function cria notificação corretamente
 - ☐ Não gera duplicatas (teste manual)
-

FASE 10 — PERFIL DO PERSONAL E WHITE-LABEL WEB

Objetivo: Personal configura perfil completo e identidade visual da plataforma.

Duração: 2–3 dias

SPECs: PROF-01 a PROF-04, WL-04

ETAPA 10.1 – Configurações do Personal

Tarefas

- ☐ Criar rota `/dashboard/settings` com abas: Perfil, Aparência, Plano
- ☐ **Aba Perfil:** nome, CREF, telefone, Instagram, WhatsApp – com validação e save
- ☐ **Aba Aparência:** nome do app, cor primária (color picker), logo (upload), mensagem de boas-vindas
 - Preview em tempo real da cor primária (SPEC-PROF-03 AC-03)
 - Upload de logo com preview
 - Campos bloqueados no plano Free com CTA de upgrade (SPEC-WL-04)
- ☐ **Aba Plano:** exibir plano atual, uso (alunos), data de renovação, botão upgrade (SPEC-PROF-04)
- ☐ Criar Storage Bucket `avatars` (público, máx 2MB)
- ☐ Upload de foto de perfil/logo com preview (SPEC-PROF-02)

Gate 10.1

- ☐ SPEC-PROF-01 AC-01 a AC-05 verificados
- ☐ SPEC-PROF-02 AC-01 a AC-05 verificados
- ☐ SPEC-PROF-03 AC-01 a AC-05 verificados
- ☐ SPEC-PROF-04 AC-01 a AC-06 verificados
- ☐ SPEC-WL-04 AC-01 a AC-04 verificados

FASE 11 – TESTES, QA E HARDENING WEB

Objetivo: Plataforma web robusta, segura e sem regressões antes de iniciar mobile.

Duração: 5–6 dias

SPECs: Todas as SPECs web revisitadas

ETAPA 11.1 – Revisão de RLS e Segurança

Tarefas

- ☐ Testar isolamento multi-tenant: criar 2 personals distintos e verificar que dados não se cruzam em nenhuma tabela
- ☐ Verificar que nenhuma tabela ficou sem RLS habilitado
- ☐ Testar que aluno não acessa rotas do dashboard (middleware)
- ☐ Revisar que `service_role_key` não está exposta em nenhum Client Component
- ☐ Verificar headers de segurança no Vercel (CSP, HSTS)
- ☐ Testar fluxo completo de convite → login → troca de senha com aluno real

Gate 11.1

- ☐ Nenhum vazamento de dados entre tenants
 - ☐ Tabelas sem RLS: nenhuma
 - ☐ `service_role_key` ausente do bundle client
-

ETAPA 11.2 – Testes de Fluxos Críticos

Tarefas

Executar manualmente (ou automatizado) os fluxos end-to-end:

- ☐ **Fluxo 1:** Cadastro do personal → confirmar e-mail → login → configurar perfil
- ☐ **Fluxo 2:** Criar aluno → enviar convite → aluno faz primeiro acesso → troca senha
- ☐ **Fluxo 3:** Criar exercício com vídeo YouTube → criar treino → adicionar exercícios → publicar
- ☐ **Fluxo 4:** Criar avaliação com 3 fotos → visualizar histórico → editar → excluir
- ☐ **Fluxo 5:** Criar fatura → enviar lembrete WhatsApp → marcar como paga
- ☐ **Fluxo 6:** Upload de arquivo PDF → visualizar na lista
- ☐ **Fluxo 7:** Inativar aluno → tentar login (deve bloquear) → reativar

Gate 11.2

- ☐ Todos os 7 fluxos passam sem erros

- ☐ Nenhum erro 500 nos logs do Vercel
-

ETAPA 11.3 – Performance e UX

Tarefas

- ☐ Lighthouse score: Performance $\geq$ 85, Accessibility $\geq$ 90
- ☐ Testar em resolução 1280px, 1024px e 768px
- ☐ Loading skeletons em todas as páginas com fetch
- ☐ Toast de erro em todas as ações que podem falhar
- ☐ Estados vazios em todas as listas
- ☐ Formulários: mensagens de erro claras em todos os campos
- ☐ Testar com conexão lenta (Chrome DevTools → Slow 3G)

Gate 11.3

- ☐ Lighthouse sem alertas críticos
 - ☐ Nenhuma tela sem loading state
 - ☐ Nenhuma tela sem estado vazio
-

ETAPA 11.4 – Revisão Final Web

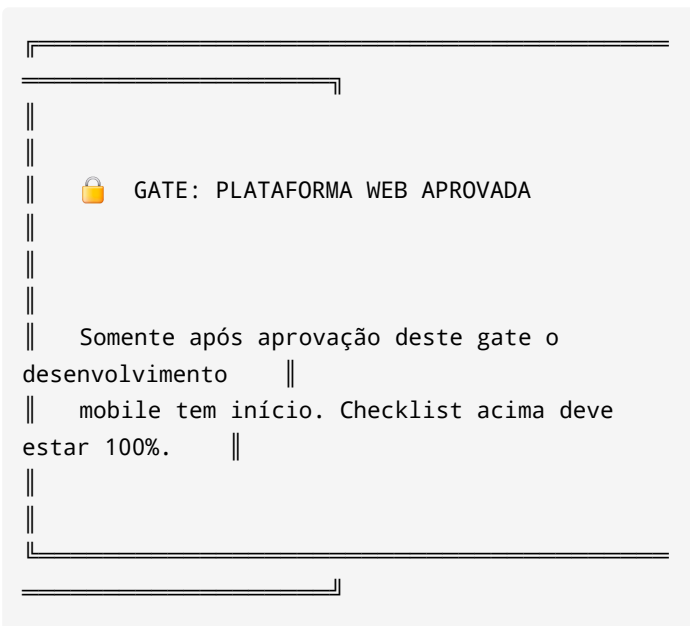
Tarefas

- ☐ Revisar SPEC completa: verificar que todos os ACs foram implementados

- ☐ Corrigir quaisquer desvios encontrados
- ☐ Atualizar documentação de variáveis de ambiente
- ☐ Tag de versão no GitHub: `git tag v1.0-web-stable`
- ☐ Confirmar deploy em produção estável

Gate 11.4 — WEB RELEASE GATE

- ☐ Todos os fluxos da ETAPA 11.2 passam
- ☐ Lighthouse ≥ 85 em performance
- ☐ Tag `v1.0-web-stable` criada
- ☐ **APROVAÇÃO FORMAL ANTES DE AVANÇAR PARA MOBILE**



FASE 12 — SETUP MOBILE

Objetivo: Ambiente React Native / Expo configurado e conectado ao Supabase existente.

Duração: 2 dias

SPECs: Nenhuma (pré-requisito técnico)

ETAPA 12.1 — Configuração do Projeto Expo

Tarefas

- [] Criar projeto: `npx create-expo-app fitcoach-mobile --template expo-template-blank-typescript`
- [] Instalar Expo Router: `npx expo install expo-router`
- [] Instalar dependências:

```
@supabase/supabase-js
expo-secure-store
expo-image-picker
expo-document-picker
expo-av (player de vídeo)
expo-notifications
expo-local-authentication (biometria)
zustand
react-native-reanimated
react-native-gesture-handler
```

```
@shopify/flash-list  
react-native-gifted-charts
```

- [] Configurar `app.json`: nome, slug, scheme (`fitcoachpro://`), ícone placeholder
- [] Criar `.env` com `EXPO_PUBLIC_SUPABASE_URL` e `EXPO_PUBLIC_SUPABASE_ANON_KEY`
- [] Criar `src/lib/supabase.ts` com cliente Supabase usando `expo-secure-store`
- [] Criar estrutura de pastas: `src/app/`, `src/components/`, `src/lib/`, `src/hooks/`, `src/store/`
- [] Testar `npx expo start` no dispositivo físico (ou emulador)

Gate 12.1

- [] App abre no dispositivo sem erros
- [] Conexão com Supabase funciona (teste manual de query simples)
- [] Expo Router configurado com rotas básicas

ETAPA 12.2 – Design System Mobile

Tarefas

- [] Criar `src/constants/theme.ts` com tokens: cores, tipografia, espaçamento, border radius
- [] Criar componentes base: `<Button />`, `<Input />`, `<Card />`, `<Badge />`, `<Avatar />`

- [] Criar `<AppHeader />` com logo dinâmico (carrega dados do personal do aluno logado)
- [] Criar `<BottomNav />` com tabs: Início, Instagram, WhatsApp, Menu
- [] Criar tela de playground `src/app/dev/design.tsx` (somente em `__DEV__`)
- [] Implementar sistema de tema dinâmico: cor primária carregada do perfil do personal

Gate 12.2

- [] Componentes base funcionam sem erros no dispositivo
 - [] Cor primária muda dinamicamente via prop/context
-

FASE 13 – AUTENTICAÇÃO MOBILE

Objetivo: Aluno consegue fazer login, trocar senha no primeiro acesso e manter sessão.

Duração: 2–3 dias

SPECs: AUTH-04, AUTH-05, AUTH-06

ETAPA 13.1 – Tela de Login do Aluno

Tarefas

- ☐ Criar rota `src/app/(auth)/login.tsx`
- ☐ Campos: e-mail, senha (com toggle de visibilidade)
- ☐ Integrar `supabase.auth.signInWithPassword()`
- ☐ Após login: verificar metadata `must_change_password`
 - Se `true` → redirecionar para `/change-password`
 - Se `false` → redirecionar para Home
- ☐ Verificar se aluno está ativo (`students.is_active`) após login (SPEC-AUTH-05 AC-03)
- ☐ Mensagem de erro genérica para credenciais inválidas
- ☐ Loading state no botão

Gate 13.1

- ☐ SPEC-AUTH-05 AC-01 a AC-05 verificados
- ☐ Aluno inativo vê mensagem correta e não entra

ETAPA 13.2 – Primeiro Acesso e Troca de Senha

Tarefas

- ☐ Criar rota `src/app/(auth)/change-password.tsx`
- ☐ Campos: nova senha, confirmar senha
- ☐ Validar: nova senha $\neq$ senha temporária, confirmação idêntica, 8+ chars

- [] `supabase.auth.updateUser({ password: newPassword })`
- [] Atualizar metadata: `must_change_password = false`
- [] Redirecionar para Home após troca
- [] Bloqueio de navegação: aluno não pode ir para outra tela antes de trocar (SPEC-AUTH-04 AC-02)

Gate 13.2

- [] SPEC-AUTH-04 AC-01 a AC-06 verificados
- [] Não é possível navegar sem trocar senha no primeiro acesso

ETAPA 13.3 — Sessão Persistente e Logout

Tarefas

- [] Configurar `supabase.auth` com `storage: ExpoSecureStore` para persistência
- [] Sessão persiste 30 dias sem novo login (SPEC-AUTH-05 AC-02)
- [] Implementar listener `supabase.auth.onAuthStateChange` no root layout
- [] Logout: `supabase.auth.signOut()` + limpar Zustand stores + redirect login (SPEC-AUTH-06)
- [] Oferecer biometria após primeiro login bem-sucedido (SPEC-AUTH-05 AC-05)

- ☐ Proteção de rotas: `(student)` layout verifica sessão

Gate 13.3

- ☐ SPEC-AUTH-06 AC-01 a AC-04 verificados
 - ☐ Sessão persiste após fechar e reabrir o app
 - ☐ Logout limpa cache e redireciona
-

FASE 14 — HOME E FREQUÊNCIA SEMANAL

Objetivo: Aluno vê a tela Home com saudação personalizada e frequência da semana.

Duração: 2–3 dias

SPECs: FRQ-01, WL-01, WL-02, WL-03

ETAPA 14.1 — Contexto do Personal (Tenant)

Tarefas

- ☐ Criar hook `usePersonalContext()`: carrega dados do personal do aluno logado
- ☐ Dados: `app_name`, `primary_color`, `logo_url`, `cref`, `instagram_url`, `whatsapp_number`, `welcome_message`

- [] Criar Context Provider `PersonalContext` no root layout `(student)`
- [] Aplicar `primary_color` dinamicamente nos componentes

Gate 14.1

- [] Cor primária do personal é aplicada nos botões e destaques
 - [] Logo do personal aparece no header
-

ETAPA 14.2 – Tela Home

Tarefas

- [] Criar `src/app/(student)/index.tsx`
- [] Header: logo/nome do personal, CREF
- [] Saudação dinâmica por horário: "Bom dia/Boa tarde/Boa noite, [Nome]!"
- [] Mensagem de boas-vindas do personal
- [] Widget `<WeeklyFrequency />`:
 - Buscar sessões da semana atual (domingo–sábado)
 - 7 círculos com letras S T Q Q S S D
 - Dia com treino: círculo preenchido + check (cor primária do personal)
 - Dia atual: borda destacada

- ☐ Grid de atalhos 2x3: Treinos, Treinos Extras, Avaliações, Meu Progresso, Faturas, Arquivos
- ☐ BottomNav: Início, Instagram (link externo), WhatsApp (link externo), Menu
- ☐ Ocultar ícone se Instagram/WhatsApp não cadastrado (SPEC-WL-02 AC-03 e AC-04)

Gate 14.2

- ☐ SPEC-FRQ-01 AC-01 a AC-06 verificados
 - ☐ SPEC-WL-01 AC-01 a AC-06 verificados
 - ☐ SPEC-WL-02 AC-01 a AC-04 verificados
 - ☐ SPEC-WL-03 AC-01 a AC-03 verificados
 - ☐ Widget de frequência atualiza ao finalizar sessão de treino
-

FASE 15 – TREINOS (VISUALIZAÇÃO MOBILE)

Objetivo: Aluno visualiza seus treinos e exercícios com vídeos antes de iniciar.

Duração: 3–4 dias

SPECs: TRE-07, TRE-08, EXE-05

ETAPA 15.1 – Lista de Treinos

Tarefas

- ☐ Criar `src/app/(student)/workouts/index.tsx`
- ☐ Criar hook `useWorkouts()` – carrega planos publicados do aluno
- ☐ Componente `<WorkoutCard />` :
 - Nome e descrição/foco muscular
 - "Executado [N] vezes, última execução em DD/MM/AAAA" (da VIEW `workout_plan_stats`)
 - Botões: Evolução, Feedbacks, Ver treino
- ☐ Seção separada para Treinos Extras
- ☐ Estado vazio: "Seu personal ainda não publicou treinos para você."
- ☐ Usar `FlashList` para performance em listas longas

Gate 15.1

- ☐ SPEC-TRE-07 AC-01 a AC-06 verificados
- ☐ Lista carrega dados reais do banco

ETAPA 15.2 – Tela de Visualização do Treino (Preview)

Tarefas

- ☐ Criar `src/app/(student)/workouts/[id]/preview.tsx`
- ☐ Banner azul de modo visualização

- ☐ Botão INICIAR (verde, destaque) – chama SPEC-EXET-01
- ☐ Lista de exercícios com `<ExerciseCard />` :
 - Nome (negrito), séries, carga, intervalo
 - Instruções (quando houver)
 - Thumbnail do vídeo com botão play
- ☐ Tocar no vídeo → player inline (não inicia o treino) (SPEC-TRE-08 AC-04)
- ☐ Nenhuma sessão criada no banco (SPEC-TRE-08 AC-05)

Gate 15.2

- ☐ SPEC-TRE-08 AC-01 a AC-05 verificados
- ☐ Vídeo reproduz sem criar sessão

ETAPA 15.3 – Player de Vídeo

Tarefas

- ☐ Criar componente `<VideoPlayer />` com `expo-av`
- ☐ Suporte a: URL do Storage (mp4), YouTube (WebView com player embed), Instagram (WebView)
- ☐ Controles: play/pause, barra de progresso, fullscreen
- ☐ Thumbnail como placeholder antes de carregar
- ☐ Mensagem de erro se offline (SPEC-EXE-05 AC-04)

Gate 15.3

- [] SPEC-EXE-05 AC-01 a AC-05 verificados
 - [] Vídeo do YouTube e upload funcionam no dispositivo
-

FASE 16 – EXECUÇÃO DO TREINO

Objetivo: Aluno consegue executar um treino completo registrando cargas, usar o timer e finalizar com feedback.

Duração: 5–6 dias

SPECs: EXET-01 a EXET-06, FBK-01, FBK-02

ETAPA 16.1 – Store Zustand da Sessão

Tarefas

- [] Criar `src/store/workout-session.store.ts` com Zustand
- [] Estado: `sessionId`, `plan`, `currentExerciseIndex`, `exerciseLogs`, `startedAt`, `isActive`
- [] Actions: `startSession`, `logSet`, `nextExercise`, `prevExercise`, `finishSession`, `reset`
- [] Persistência local (`AsyncStorage`) para sobreviver a fechamento acidental do app

Gate 16.1

- [] Store funciona com React DevTools / console

- [] Estado persiste ao fechar e reabrir o app
-

ETAPA 16.2 – Tela de Execução do Treino

Tarefas

- [] Criar
`src/app/(student)/workouts/[id]/active.tsx`
- [] Header: ← Sair | Nome do treino | progresso (2/5)
- [] Barra de progresso linear
- [] Componente `<SeriesTracker />`:
 - Para cada série: campo de carga editável + checkbox de conclusão
 - Pré-preencher carga sugerida pelo personal
 - Ao marcar: bloquear campo, check visual, iniciar timer
 - Desmarcar para corrigir (SPEC-EXET-02 AC-06)
- [] Componente `<RestTimer />`:
 - Countdown regressivo em MM:SS
 - Barra de progresso visual
 - Botões: Pausar / Pular
 - Vibração + som ao chegar em 0 (SPEC-EXET-03 AC-03)
 - Continua em background (SPEC-EXET-03 AC-06)
- [] Botões navegação: ← Anterior | Próximo →

- ☐ Botão "Finalizar Treino"
- ☐ Manter tela ativa (sem sleep) durante execução (SPEC-EXET-01 AC-05)
- ☐ Persistir cada série no banco imediatamente ao marcar (SPEC-EXET-02 AC-07)

Gate 16.2

- ☐ SPEC-EXET-01 AC-01 a AC-05 verificados
 - ☐ SPEC-EXET-02 AC-01 a AC-07 verificados
 - ☐ SPEC-EXET-03 AC-01 a AC-06 verificados
 - ☐ SPEC-EXET-04 AC-01 a AC-05 verificados
-

ETAPA 16.3 — Finalizar Treino e Feedback

Tarefas

- ☐ Dialog de confirmação se séries não concluídas (SPEC-EXET-05 AC-02)
- ☐ Ao confirmar: `SessionService.finish()` → tela de conclusão
- ☐ Tela de conclusão: duração, exercícios, séries realizadas
- ☐ Modal de feedback automático (SPEC-FBK-01):
 - Escala de emojis 1–5
 - Campo de observação
 - Botão "Pular"

- ☐ Após feedback (ou pular): redirecionar para lista de treinos
- ☐ Frequência semanal atualizada imediatamente na Home

Gate 16.3

- ☐ SPEC-EXET-05 AC-01 a AC-06 verificados
 - ☐ SPEC-FBK-01 AC-01 a AC-06 verificados
 - ☐ Widget de frequência na Home reflete a sessão finalizada
-

ETAPA 16.4 – Sessão Interrompida

Tarefas

- ☐ Ao abrir o app com sessão em andamento: perguntar "Continuar de onde parou?" (SPEC-EXET-05 ALT-01)
- ☐ Opção "Continuar": restaura estado do Zustand
- ☐ Opção "Descartar": marca sessão como não concluída e limpa store

Gate 16.4

- ☐ Sessão interrompida é restaurada corretamente
 - ☐ Descartar limpa o estado sem afetar histórico
-

ETAPA 16.5 – Evolução do Treino (Mobile)

Tarefas

- ☐ Criar
`src/app/(student)/workouts/[id]/evolution.tsx`
- ☐ Dropdown para selecionar exercício
- ☐ Gráfico de linha com `react-native-gifted-charts`
- ☐ Comparativo numérico de carga
- ☐ Lista de sessões com data, duração e nota
- ☐ Mensagem orientativa se menos de 2 sessões
(SPEC-EXET-06 AC-06)

Gate 16.5

- ☐ SPEC-EXET-06 AC-01 a AC-06 verificados
-

FASE 17 – AVALIAÇÕES E PROGRESSO MOBILE

Objetivo: Aluno visualiza avaliações e registra seu progresso.

Duração: 3–4 dias

SPECs: AVA-03, PRG-01, PRG-02, PRG-03

ETAPA 17.1 – Avaliações (Mobile)

Tarefas

- ☐ Criar `src/app/(student)/assessments/index.tsx`
- ☐ Lista de avaliações em ordem decrescente
- ☐ Tela de detalhe: todas as medidas + fotos (via signed URL)
- ☐ Gráficos: evolução de peso e % gordura entre avaliações
- ☐ Comparativo: "Você perdeu X kg desde a última avaliação 
- ☐ Fotos abertas em modal fullscreen com zoom

Gate 17.1

- ☐ SPEC-AVA-03 AC-01 a AC-07 verificados
- ☐ Fotos carregam via signed URL (privadas)

ETAPA 17.2 – Meu Progresso (Mobile)

Tarefas

- ☐ Criar `src/app/(student)/progress/index.tsx`
- ☐ Botão "Adicionar registro": campos peso + foto (câmera ou galeria) + nota
- ☐ Upload de foto para Storage
- ☐ Gráfico de peso ao longo do tempo
- ☐ Timeline de fotos em ordem cronológica decrescente
- ☐ Filtro por período: 1 mês, 3 meses, 6 meses, tudo
- ☐ Comparativo primeiro vs último registro

Gate 17.2

- ☐ SPEC-PRG-01 AC-01 a AC-04 verificados
 - ☐ SPEC-PRG-02 AC-01 a AC-05 verificados
-

FASE 18 — FATURAS E ARQUIVOS

MOBILE

Objetivo: Aluno visualiza faturas e acessa arquivos enviados pelo personal.

Duração: 2–3 dias

SPECs: FAT-07, ARQ-03, FBK-04

ETAPA 18.1 — Faturas (Mobile)

Tarefas

- ☐ Criar `src/app/(student)/invoices/index.tsx`
- ☐ Seção "Em aberto": faturas pendentes e vencidas em destaque
- ☐ Seção "Histórico": faturas pagas
- ☐ Botão "Pagar": abre link de pagamento no navegador
- ☐ Botão "Copiar chave PIX": copia para clipboard + toast "Chave PIX copiada!"
- ☐ Fatura vencida: destaque vermelho

Gate 18.1

- ☐ SPEC-FAT-07 AC-01 a AC-06 verificados

ETAPA 18.2 – Arquivos (Mobile)

Tarefas

- ☐ Criar `src/app/(student)/files/index.tsx`
- ☐ Lista de arquivos com ícone por tipo (PDF, imagem)
- ☐ PDF: abrir com `expo-document-picker` ou `WebView` inline
- ☐ Imagens: galeria com zoom
- ☐ Botão Download para todos os tipos
- ☐ Estado vazio se nenhum arquivo disponível

Gate 18.2

- ☐ SPEC-ARQ-03 AC-01 a AC-06 verificados
- ☐ PDF abre inline no app

ETAPA 18.3 – Feedbacks do Treino (Tela Mobile)

Tarefas

- ☐ Criar `src/app/(student)/workouts/[id]/feedbacks.tsx`
- ☐ Lista de feedbacks do aluno para o treino
- ☐ Sessões sem feedback: botão "Avaliar" (feedback retroativo – SPEC-FBK-02)
- ☐ Sessões com feedback: emoji + nota + observação

Gate 18.3

- ☐ SPEC-FBK-04 AC-01 a AC-03 verificados
 - ☐ SPEC-FBK-02 AC-01 a AC-03 verificados
-

FASE 19 – NOTIFICAÇÕES PUSH

Objetivo: Aluno e personal recebem notificações push relevantes.

Duração: 2–3 dias

SPECs: NOT-01 a NOT-06

ETAPA 19.1 – Permissão e Registro de Token

Tarefas

- ☐ Solicitar permissão de notificação na primeira abertura do app (SPEC-NOT-01 AC-01)
- ☐ Registrar token Expo Push no banco (`push_tokens`) (SPEC-NOT-01 AC-04)
- ☐ Revogar token no logout (SPEC-AUTH-06 AC-04)
- ☐ App funciona normalmente se permissão negada (SPEC-NOT-01 AC-02)

Gate 19.1

- ☐ SPEC-NOT-01 AC-01 a AC-04 verificados

- [] Token salvo no banco após permissão concedida
-

ETAPA 19.2 – Envio de Push nas Edge Functions

Tarefas

- [] Atualizar Edge Functions existentes para enviar push via Expo Push API:
 - `invite-student` : não envia push (ainda não tem token)
 - `overdue-invoices` : push para aluno (NOT-04)
 - `inactivity-alert` : push para personal (NOT-05)
 - Publicação de treino: push para aluno (NOT-03) – chamar ao publicar
 - Novo arquivo: push para aluno (NOT-06) – chamar ao fazer upload
 - Feedback enviado: push para personal (NOT-06)
- [] Integrar Expo Push Notifications API:
`https://exp.host/--/api/v2/push/send`

Gate 19.2

- [] SPEC-NOT-03 a NOT-06 AC-comuns verificados
 - [] Push chega no dispositivo para ao menos 2 gatilhos testados
-

ETAPA 19.3 – Central de Notificações (Mobile)

Tarefas

- ☐ Deep link: tocar na notificação push abre a tela correspondente (SPEC-NOT-01 AC-03)
- ☐ Criar
`src/app/(student)/notifications/index.tsx`
- ☐ Badge no ícone de menu com contagem de não lidas
- ☐ Marcar como lida ao tocar
- ☐ "Marcar todas como lidas"

Gate 19.3

- ☐ SPEC-NOT-02 AC-01 a AC-06 verificados
 - ☐ Deep link funciona para ao menos 2 tipos de notificação
-

FASE 20 – WHITE-LABEL MOBILE

Objetivo: App se adapta completamente à identidade do personal do aluno.

Duração: 1–2 dias

SPECs: WL-01 a WL-04

ETAPA 20.1 – Aplicação Completa do White-label

Tarefas

- ☐ Verificar que cor primária está aplicada em TODOS os elementos definidos (SPEC-WL-01 AC-02): botões, ícones ativos da BottomNav, checkboxes de frequência, badges
- ☐ Fallback de logo: iniciais em círculo colorido (SPEC-WL-01 AC-05)
- ☐ Mudanças no perfil do personal refletem sem necessidade de atualizar app (SPEC-WL-01 AC-06) – dados carregados do banco a cada abertura
- ☐ Testar com 2 personals de cor diferente (azul vs verde) usando 2 contas de aluno

Gate 20.1

- ☐ SPEC-WL-01 AC-01 a AC-06 verificados
 - ☐ SPEC-WL-04 AC-01 a AC-04 verificados (plano Free usa branding padrão)
 - ☐ Teste com 2 tenants distintos sem contaminação visual
-
-

FASE 21 – TESTES, QA E DEPLOY MOBILE

Objetivo: App mobile robusto, testado em dispositivos reais e pronto para entrega.

Duração: 4–5 dias

ETAPA 21.1 — Testes de Fluxos Críticos Mobile

Tarefas

Executar em dispositivo físico Android (mínimo) e iOS (se disponível):

- ☐ **Fluxo 1:** Login com credenciais do convite → trocar senha → acessar Home
- ☐ **Fluxo 2:** Visualizar treino em preview → iniciar → registrar séries → usar timer → finalizar → feedback
- ☐ **Fluxo 3:** Fechar app durante treino → reabrir → continuar de onde parou
- ☐ **Fluxo 4:** Visualizar avaliações e gráficos de evolução
- ☐ **Fluxo 5:** Registrar progresso com foto
- ☐ **Fluxo 6:** Visualizar fatura e copiar chave PIX
- ☐ **Fluxo 7:** Receber e visualizar arquivo PDF
- ☐ **Fluxo 8:** Receber notificação push e navegar pelo deep link

Gate 21.1

- ☐ Todos os 8 fluxos passam em dispositivo Android real

- ☐ Nenhum crash registrado
-

ETAPA 21.2 – Performance e Qualidade Mobile

Tarefas

- ☐ Testar em conexão 3G lenta (modo avião + dados baixos)
- ☐ Verificar offline: treino previamente carregado pode ser visualizado sem internet
- ☐ Verificar que a tela não dorme durante execução do treino
- ☐ Testar timer em background (app minimizado)
- ☐ Verificar que logout limpa todos os dados do Zustand
- ☐ Testar isolamento: aluno A não vê dados do aluno B

Gate 21.2

- ☐ App funciona em 3G sem travamentos críticos
 - ☐ Isolamento multi-tenant confirmado no mobile
-

ETAPA 21.3 – Build e Deploy

Tarefas

- ☐ Configurar EAS Build: `eas build:configure`

- ☐ Gerar build de preview (APK) para Android: `eas build --platform android --profile preview`
- ☐ Distribuir APK para testes via link EAS
- ☐ Configurar OTA updates (Expo Updates) para atualizações sem nova build
- ☐ Tag de versão: `git tag v1.0-mobile-stable`

Gate 21.3 — 🚩 **RELEASE GATE FINAL** ✅

- ☐ APK instalável e funcional em dispositivo físico
- ☐ Todos os fluxos da ETAPA 21.1 passam na build de produção
- ☐ Tag `v1.0-mobile-stable` criada
- ☐ **PRODUTO PRONTO PARA ENTREGA AO PRIMEIRO PERSONAL BETA**

RESUMO EXECUTIVO

Fase	Descrição	Semana	Status
0	Infraestrutura e Setup	1	☐
1	Autenticação Web	1-2	☐
2	Gestão de Alunos	2-3	☐
3	Banco de Exercícios	3	☐
4	Criação de Treinos	4-5	☐

Fase	Descrição	Semana	Status
5	Avaliações Físicas	5-6	☐
6	Financeiro / Faturas	6-7	☐
7	Arquivos	7	☐
8	Dashboard e Relatórios	8	☐
9	Notificações Web	8-9	☐
10	White-label e Config Web	9	☐
11	QA e Hardening Web	10	☐
	GATE: WEB APROVADA	10	☐
12	Setup Mobile	11	☐
13	Autenticação Mobile	11	☐
14	Home e Frequência	12	☐
15	Treinos (Visualização)	12-13	☐
16	Execução do Treino	13-14	☐
17	Avaliações e Progresso	14-15	☐
18	Faturas e Arquivos Mobile	15	☐
19	Notificações Push	15-16	☐
20	White-label Mobile	16	☐
21	QA e Deploy Mobile	17	☐

Fase	Descrição	Semana	Status
	RELEASE v1.0	17	

COMO USAR ESTE DOCUMENTO

1. **Execute uma etapa por vez** — não pule etapas, não adiante fases
2. **Marque o gate antes de avançar** — gate parcial = não avançar
3. **Toda task tem uma SPEC de referência** — em caso de dúvida, consulte a SPEC
4. **Banco sempre primeiro** — dentro de cada fase, sempre começa pela etapa de banco
5. **Deploy ao final de cada fase** — nunca acumular código sem subir para Vercel/EAS

Documento gerado em 17/06/2026 | 21 fases · 60 etapas · 17 semanas estimadas